

From Good to Great

Prompt Optimisation for Official Statistics

Systematic improvement rather than trial and error: evaluation sets, quality metrics, variance, token cost, caching, and when to move beyond prompting.



GOOD



GREAT

What you will be able to do



Build an evaluation set

that represents your real data, languages and edge cases.



Measure output quality

with a metric that matches the task, not by eye.



Control variance

so the same input gives the same, defensible output.



Manage context and cost

estimate tokens, use caching and batching.



Choose the right approach

between prompting, RAG and fine-tuning.

SESSION FORMAT



Talk

Concepts and worked examples



Lab

Optimise a coding prompt on a frozen evaluation set



Wrap-up

Best scores and hand-over to the Groq benchmark

Six steps from a good prompt to a great one

01**From craft to method**

Why “it looked fine” is not a quality standard for official statistics.

02**Build an evaluation set**

The frozen test that tells you whether a change helped.

03**Measure quality**

Metrics per task type, scorecards and LLM-as-judge.

04**Control variance**

Why answers change between runs, and the levers to stop it.

05**Context, tokens & cost**

Estimate the bill, shorten context, cache and batch.

06**Prompt, RAG or fine-tune?**

Match the tool to the failure you observe.

Then: hands-on lab · pitfalls for statistical offices · the great-prompt checklist

SECTION 1 OF 6

01

From craft to method

Why does a prompt that worked on yesterday's bulletin fail on today's?



“It looked fine” is not a quality standard

A “GOOD” PROMPT

Tested on	✗ the one example you happened to try
Judged by	✗ reading the answer and feeling satisfied
Error rate	✗ unknown
Between runs	✗ answers drift without warning
Cost	✗ discovered on the invoice

A “GREAT” PROMPT

✓ a frozen set of 30–50 representative cases
✓ written criteria and a numeric score
✓ measured, with named failure modes
✓ stable, pinned model, versioned prompt
✓ estimated per 1,000 documents in advance



Official statistics raise the bar. The UN Fundamental Principles ask offices to choose methods on professional and scientific grounds (Principle 2) and to protect individual data (Principle 6). An LLM step in production must meet the same test.

Source: UN General Assembly Resolution 68/261, *Fundamental Principles of Official Statistics* (2014).

The six building blocks, on a real NSO task

PROMPT v0 · OCCUPATION CODING

ROLE

You are a statistical coder at the National Statistics Office.

OBJECTIVE

Assign one ISCO-08 four-digit unit group code to each job description.

CONTEXT

Descriptions are free text from the Labour Force Survey, in French or English, typed by enumerators.

RULES

Use only codes from the attached ISCO-08 list. Code the main job. If unclear, return UNCODABLE; never guess.

FORMAT

Return JSON only: {"id", "isco_code", "confidence": "high|medium|low"}

EXAMPLES

“Vendeuse de tomates au marché” → 5211 · “Moto-taxi driver” → 8321



This morning:

how do I write it?

This afternoon:

**how do I prove it works,
and make it better?**

Prompt v0 is your starting point in the lab.

Codes: ILO, International Standard Classification of Occupations (ISCO-08). 5211 = stall and market salespersons; 8321 = motorcycle drivers.

SECTION 2 OF 6

02

Build an evaluation set

How will you know that version 2 is better than version 1?



The optimisation loop



 **Change one variable at a time.** If you change three things and the score rises, you have learnt nothing about which one helped.

What an evaluation set looks like

ID	Input (as typed by the enumerator)	Gold answer	Case type
E01	<i>Vendeuse de tomates au marché central</i>	5211	Typical
E02	<i>Moto-taxi driver, owns the motorcycle</i>	8321	Typical
E03	<i>Agent de saisie à l'institut de statistique</i>	4132	Typical · FR
E04	<i>Primary teacher, sells airtime at weekends</i>	2341	Edge · 2 jobs
E05	<i>Ingénieur</i>	UNCODABLE	Edge · vague
E06	<i>works</i>	UNCODABLE	Negative

Illustrative extract. ISCO-08 codes: 5211, 8321, 4132, 2341.

A PRACTICAL STARTING MIX

~60% **Typical cases**
Everyday path, every language



~25% **Edge cases**
Where decision rules are tested



~15% **Negative cases**
The model must refuse or flag



Rule of thumb: start with 30–50 cases, then add every new failure you meet in production.

Five rules for a set you can trust



Representative

Cover every language, region, data source and format you meet in production, including poor scans and typos.



Expert-labelled

Gold answers come from subject-matter experts. Double-code, and resolve disagreements before scoring.



Frozen and versioned

Never edit a case once scoring has started. Store the set in Git with a version number.



Never in the prompt

Examples in the prompt come from a separate pool. Otherwise you are grading the model on the answers you gave it.



Grows from failures

Every error found in production becomes a new test case in the next version of the set.

SECTION 3 OF 6

03

Measure quality

Score it, don't sense it.

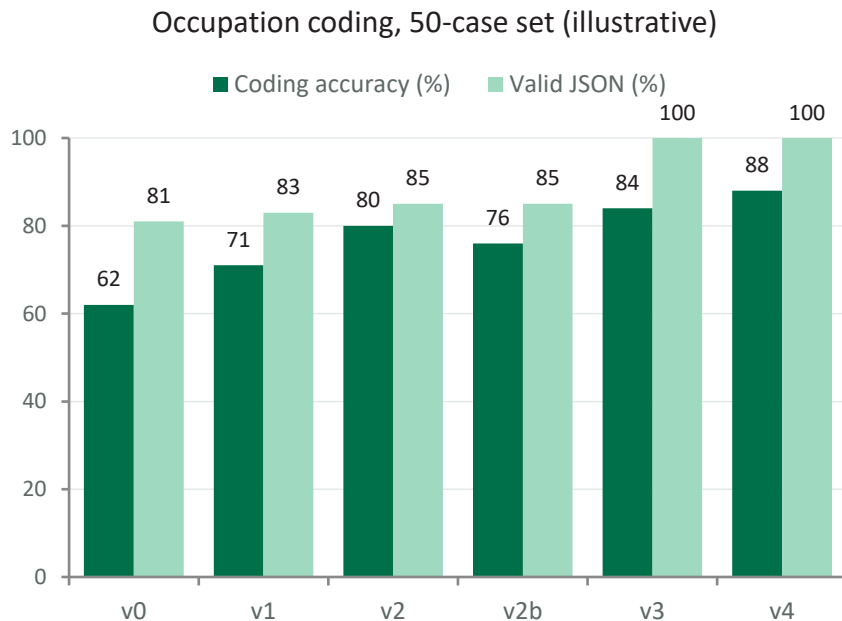


Choose the metric that matches the task

Task type	Example in an NSO	Metric	How it is scored
 Extraction	CPI figures from a PDF bulletin	Field-level exact match	Automatic: script vs. gold
 Coding	ISCO, ISIC or COICOP coding of survey text	Accuracy + per-class precision / recall	Automatic + confusion matrix
 Structured output	JSON feeding a public dashboard	Schema validity rate	Automatic: JSON Schema validator
 Grounded Q&A (RAG)	Questions on methodology notes	Faithfulness, citation accuracy	Rubric: human or calibrated judge
 Drafting	Press release, methodological note	Rubric 1–5: accuracy, completeness, tone, length	Human reviewers, calibrated judge

 Green rows: fully automatic, cheap and objective. Start there.

Keep a scorecard for every version

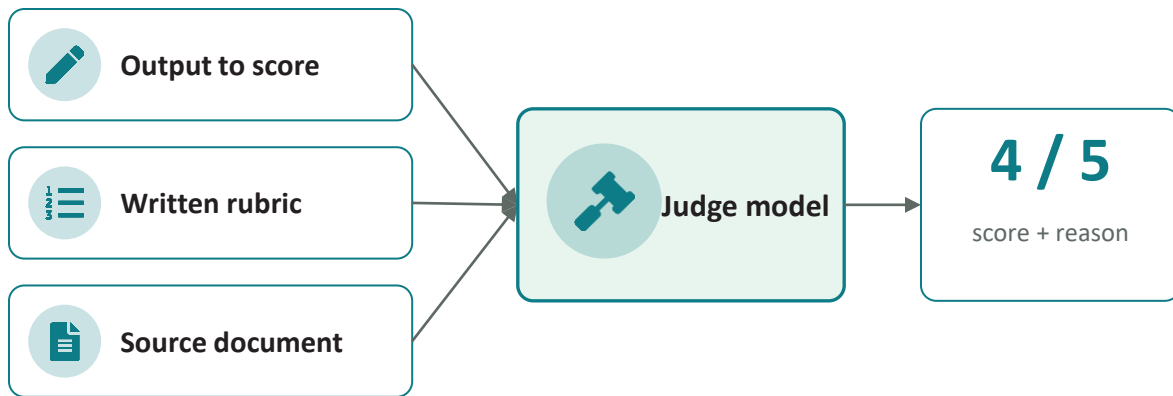


ITERATION LOG

v0	Baseline from this morning	62%	BASE
v1	Added “code the main job” rule	71%	KEEP
v2	Added 3 edge-case examples	80%	KEEP
v2b	Added 15 extra rules	76%	REJECT
v3	Enforced JSON schema	84%	KEEP
v4	Clarified UNCODABLE criteria	88%	KEEP

Illustrative figures for teaching purposes. Build your own scorecard from your evaluation set.

Using an LLM as a judge, safely



CALIBRATE BEFORE YOU TRUST

- 1 Humans score 20–30 outputs
- 2 Judge scores the same outputs
- 3 Use the judge only if agreement is high

KNOWN JUDGE BIASES

⚠ Position

Favours the answer shown first. Swap order and average.

⚠ Verbosity

Favours longer answers. Put length in the rubric.

⚠ Self-preference

Favours its own style. Use a different model family.

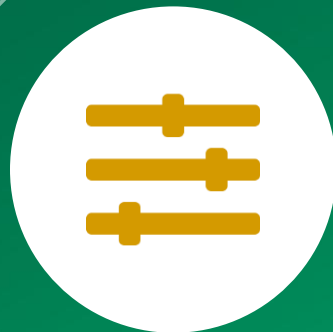
Source: Zheng et al. (2023), “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”, *NeurIPS Datasets & Benchmarks*.

SECTION 4 OF 6

04

Control variance

Same prompt, same input: why a different answer?



Where variance comes from



Sampling randomness

Temperature and top-p let the model choose among likely next tokens.



Ambiguous instructions

An unclear rule is resolved differently from one run to the next.



Silent model updates

A generic model alias can point to a new version without notice.



Input variation

Language, layout, scan quality and typos all shift the output.

THE 5-RUN CONSISTENCY TEST

E01	5211	5211	5211	5211	5211	100%
E04	2341	2341	5249	2341	5249	60%
E05	UNC	UNC	UNC	UNC	UNC	100%

Agreement rate = share of runs giving the most frequent answer. Anything below 100% on a coding task deserves a look.

Illustrative runs. UNC = UNCODABLE. 5249 = sales workers not elsewhere classified (a plausible mis-code for “sells airtime”).

Six levers to make outputs reproducible

MODEL SETTINGS



Low temperature

0 to 0.2 for extraction and coding; higher only for creative drafting.



Pin the model version

Use the dated model identifier, and record it in the log.



Cap the output

Set max tokens and a word limit; long answers wander.

PROMPT DESIGN



Constrain the format

JSON schema with a closed list of allowed values.



Write tie-break rules

"If two jobs, code the one with the most hours."



Vote on hard cases

Run 3–5 times, keep the majority, send disagreements to a human.



Temperature 0 reduces randomness; it does not guarantee identical outputs. Always measure agreement on your evaluation set.

Self-consistency: Wang et al. (2022), "Self-Consistency Improves Chain of Thought Reasoning in Language Models", arXiv:2203.11171.

SECTION 5 OF 6

05

Context, tokens and cost

What would it cost to code 100,000 survey answers?



Tokens: the unit of context and of cost

HOW A MODEL READS A SENTENCE (ILLUSTRATIVE SPLIT)

The consumer price index rose 2 . 4 % in June

11 tokens

≈ 4

characters per token

in English, or about ¾ of a word (OpenAI rule of thumb).

More

tokens per word

in many languages other than English: budget extra.

In + Out

both are billed

and output tokens are usually priced higher than input tokens.

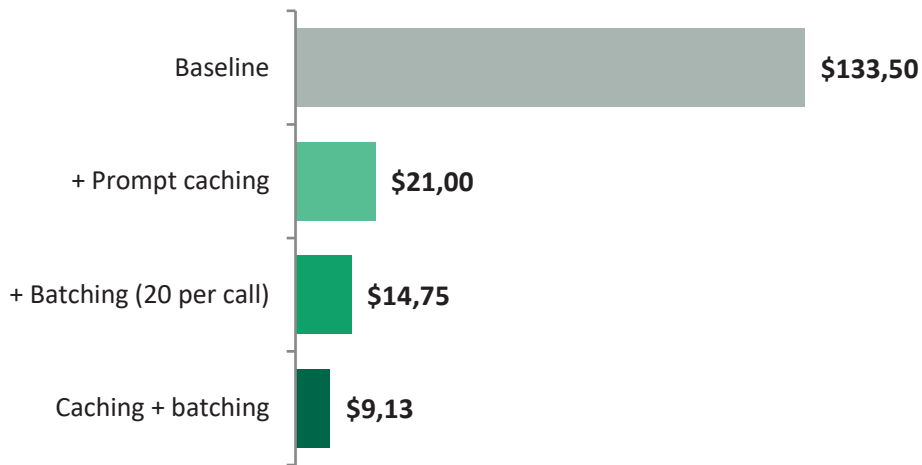
Cost = input tokens × input price + output tokens × output price

Context window ≥ instructions + examples + document + answer

Sources: OpenAI Help Center, "What are tokens and how to count them?"; Petrov et al. (2023), NeurIPS.

Worked example: coding 100,000 job descriptions

Cost to code 100,000 descriptions (illustrative prices)



ASSUMPTIONS PER CALL

2,500

fixed tokens: instructions, ISCO
extract, examples

50

tokens per job description

40

output tokens per coded answer

\$0.50 / \$1.50

per million input / output tokens

90%

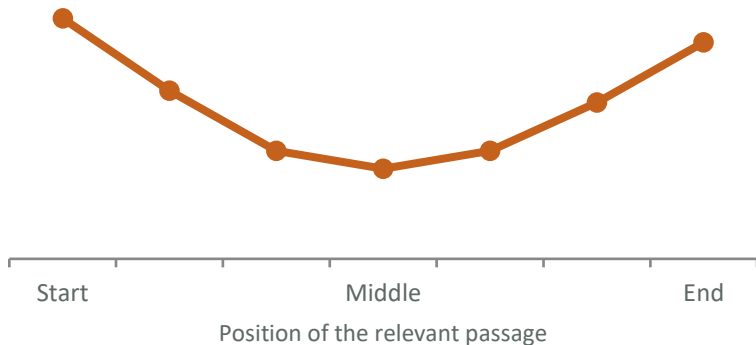
discount on cached input (assumed)

≈ **15× cheaper** through prompt design alone. Re-test accuracy after batching: more items per call can mean more errors.

Illustrative prices and discount, not a provider tariff. Cache-write surcharges ignored.

Less context, better placed

Use of a fact vs. its position in a long context
(schematic)



Schematic after Liu et al. (2023): not their exact figures.



Send only what is relevant

Pre-select pages or chunks (RAG) instead of whole reports.



Summarise or chunk

Process long documents section by section, then combine.



Place key rules at the edges

Critical instructions first; restate the key rule at the end.



Limit the answer

Word limits and max tokens cut cost and drift.

Source: Liu et al. (2023), "Lost in the Middle: How Language Models Use Long Contexts", Transactions of the ACL (2024).

Two kinds of caching



Prompt caching

at the provider

STRUCTURE EVERY CALL LIKE THIS

Instructions · ISCO list · examples

Case

stable: cached, discounted

changes

- Stable content first, variable content last.
- A date or ID at the top breaks the cache.
- Discounts, minimum size and lifetime vary by provider.



Response caching

in your own pipeline

KEY = HASH OF

model version

prompt version

input text



Instant and free on repeat requests



Perfectly reproducible answers



An audit trail for every published figure

Check each provider's documentation for current prompt-caching terms.

SECTION 6 OF 6

06

Prompt, RAG or fine-tune?

Which tool fixes which failure?

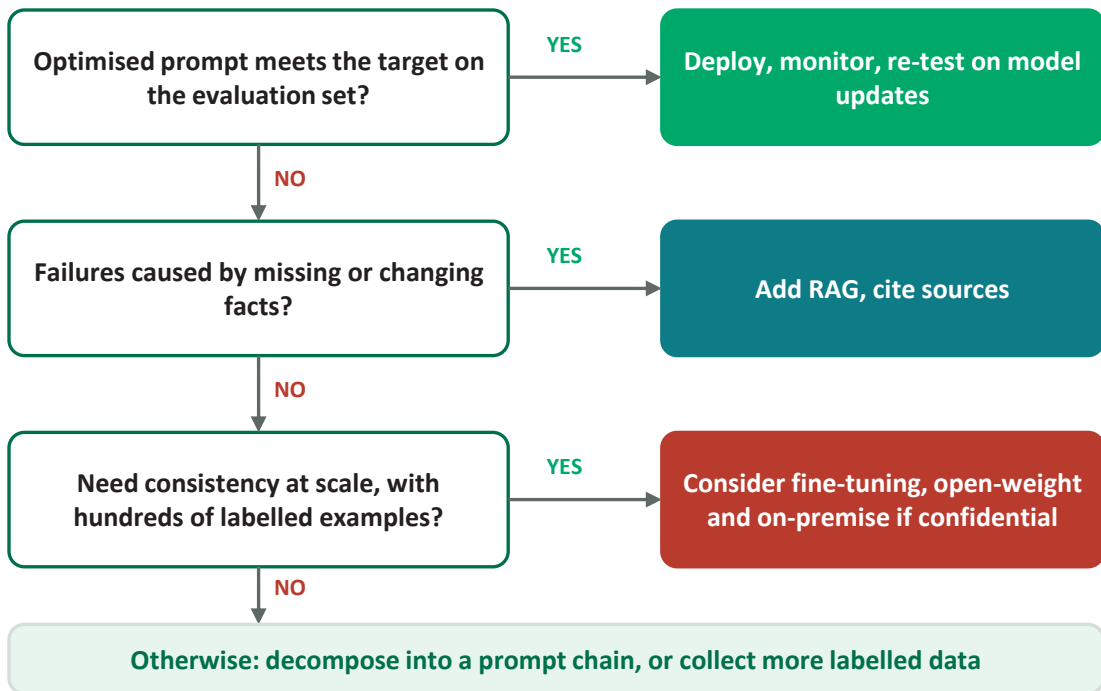


Three tools, three different problems

	 Prompt engineering START HERE	 RAG ADD KNOWLEDGE	 Fine-tuning CHANGE THE MODEL
Fixes	Behaviour you can describe in words	Missing, changing or citable knowledge	Consistent behaviour on a narrow, high-volume task
Data needed	A few examples	A document corpus	Hundreds to thousands of labelled pairs
First result	Hours	Days	Weeks
Cost profile	Low: pay per call	Moderate: embeddings + vector store	Higher: training + hosting
To update	Edit the prompt	Re-index the documents	Retrain
NSO example	<i>Drafting a press release</i>	<i>Q&A on methodology notes and metadata</i>	<i>National-scale ISCO coding with a small on-premise model</i>

Durations and data volumes are indicative orders of magnitude.

A decision path driven by your evaluation results



Sovereignty check

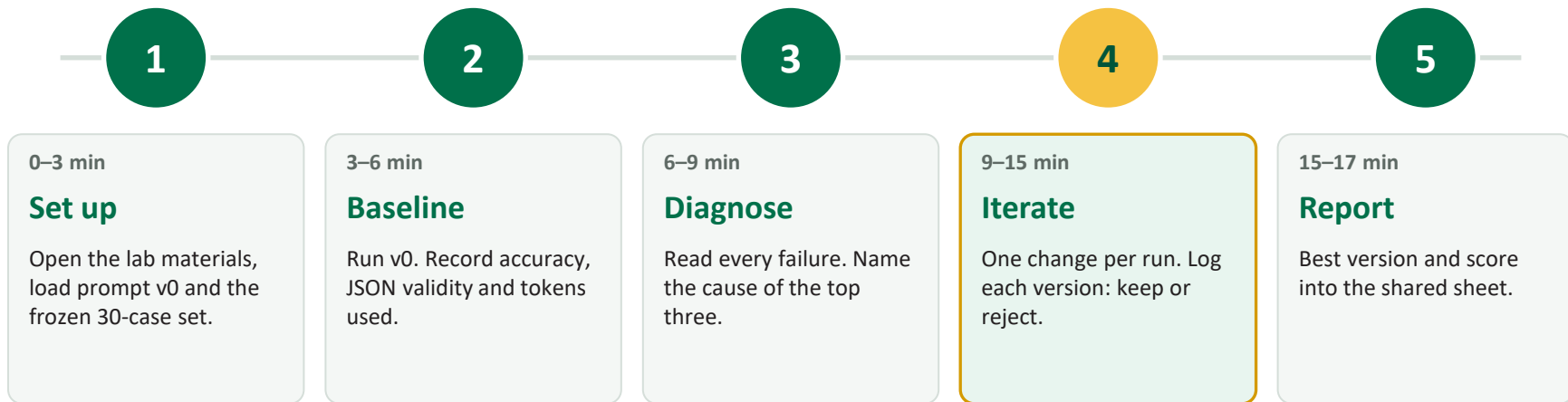
Can these data leave the office?

If not, only providers under your legal control, or self-hosted open-weight models, are eligible, whatever the route.

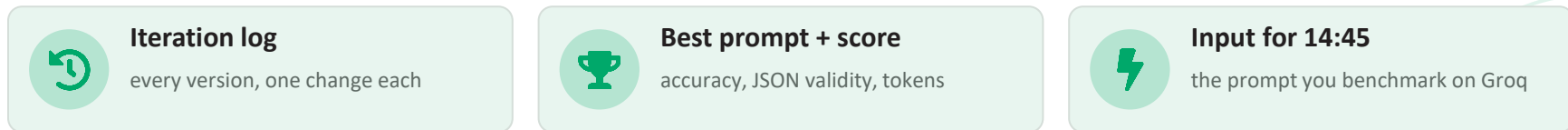
Whatever the route:

re-run the same evaluation set and log the result.

Optimise the coding prompt on a frozen set



WHAT EACH PAIR HANDS IN



The great-prompt checklist

Evaluation



Success criteria and target score written down



Frozen, versioned set of 30–50 expert-labelled cases

Quality



Metric matches the task type



Iteration log with one change per version

Variance



Model version pinned; temperature set deliberately



Agreement across repeated runs measured

Cost



Cost per 1,000 documents estimated



Stable content first, so it can be cached

Governance



Confidentiality and data terms checked



Outputs verified; prompt has a version and an owner

Key takeaways

01



Measure before you change

A frozen evaluation set turns opinions into numbers.

02



Engineer reproducibility

Pin, constrain, test repeated runs, verify before publishing.

03



Design for cost

Prompt structure, caching and batching can cut the bill by an order of magnitude.

FROM GOOD TO GREAT

**A good prompt gets an answer.
A great prompt gets an answer
you can defend.**

Thank you. Questions and discussion.

STG17 Technical Workshop · Emerging Issues, Emerging Practice



Sources



WORKSHOP

STG17 Workshop Agenda, “Emerging Issues, Emerging Practice: Innovating the Data Value Chain”, version 01/09/2026. AfDB, AU STATAFRIC, NISR.

PRINCIPLES

United Nations General Assembly (2014). Resolution 68/261, Fundamental Principles of Official Statistics.

CLASSIFICATION

International Labour Organization (2012). International Standard Classification of Occupations, ISCO-08.

LLM JUDGES

Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS Datasets and Benchmarks Track.

VARIANCE

Wang, X. et al. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171 (ICLR 2023).

TOKENS

OpenAI Help Center. What are tokens and how to count them?

LANGUAGES

Petrov, A. et al. (2023). Language Model Tokenizers Introduce Unfairness Between Languages. NeurIPS 2023.

CONTEXT

Liu, N. F. et al. (2023). Lost in the Middle: How Language Models Use Long Contexts. Transactions of the ACL, 12 (2024).

Figures marked “illustrative” or “schematic” are teaching examples, not measured results.